

INTERIO JUNCTION CRM · SECURITY DOSSIER

Security Architecture & Controls

The four applications — Customer Mobile, Customer Web, Staff Mobile, Staff Web — on a shared FastAPI + PostgreSQL backend.

Prepared for security review (CISO discussion)
Version 1.0 · 25 July 2026 · **Confidential**

Architecture · Controls · Plain-English rationale · CISO question set

Interio Junction operates **four applications** — a customer mobile app and web portal, and a staff mobile app and web CRM — on **one shared, security-hardened backend** (FastAPI) and **one database** (PostgreSQL). Security is built in depth: at the device/browser, in transit, at the API, and in the database, plus automated checks on every code change.

Posture at a glance

- **Customers and staff are fully isolated** — separate, non-interchangeable login tokens (dual-BFF); neither can reach the other's data.
- **Strong identity** — passwordless phone OTP for customers; password + MFA and phishing-resistant passkeys for staff.
- **Data protected** — TLS in transit, field-level encryption of contact details at rest, signed/expiring file links, least-privilege DB role.
- **Insider & fraud controls** — role-based access, four-eyes on money, mass-read alerts, and a tamper-evident (hash-chained) audit log.
- **Secure by build** — automated SAST, dependency/secret scanning, SBOM and authorization tests gate every change.

Honest current state. The *application and backend* controls are implemented and tested. What remains is largely **production configuration and infrastructure**: turning on HTTPS (which also unlocks passkeys), enforcing MFA, moving the encryption key to a managed KMS, and hardening the hosting. These are tracked as 4 items and summarised as a 90-day roadmap in §06.

What we're seeking from this review: an experienced steer on our top real-world risks, confirmation we're prioritising correctly for a company our size, and the two or three highest-value improvements to make next (§07).

By the numbers

4 applications, 1 backend, 1 database

30+ distinct security controls

7 control domains

2-factor + passkeys for staff

0 shared sessions across the customer/staff boundary

Interio Junction runs **four front-end applications** that all talk to **one backend** and **one database**. Customers and staff are kept strictly apart at every layer.

The four applications

- **Customer Mobile** — the Flutter app clients install on their phone.
- **Customer Web** — the browser portal for the same clients.
- **Staff Mobile** — the Flutter app for the Interio Junction team.
- **Staff Web** — the browser CRM for the team.

Shared backend & data

- **API (FastAPI)** — every request goes through it; it enforces identity, permissions and business rules.
- **PostgreSQL** — one database, accessed only by the API through a least-privilege role.

The single most important boundary

The backend is a **dual-BFF** (Backend-for-Frontend): customer requests and staff requests use *different, non-interchangeable* login tokens.

Customer token → only /api/client/*

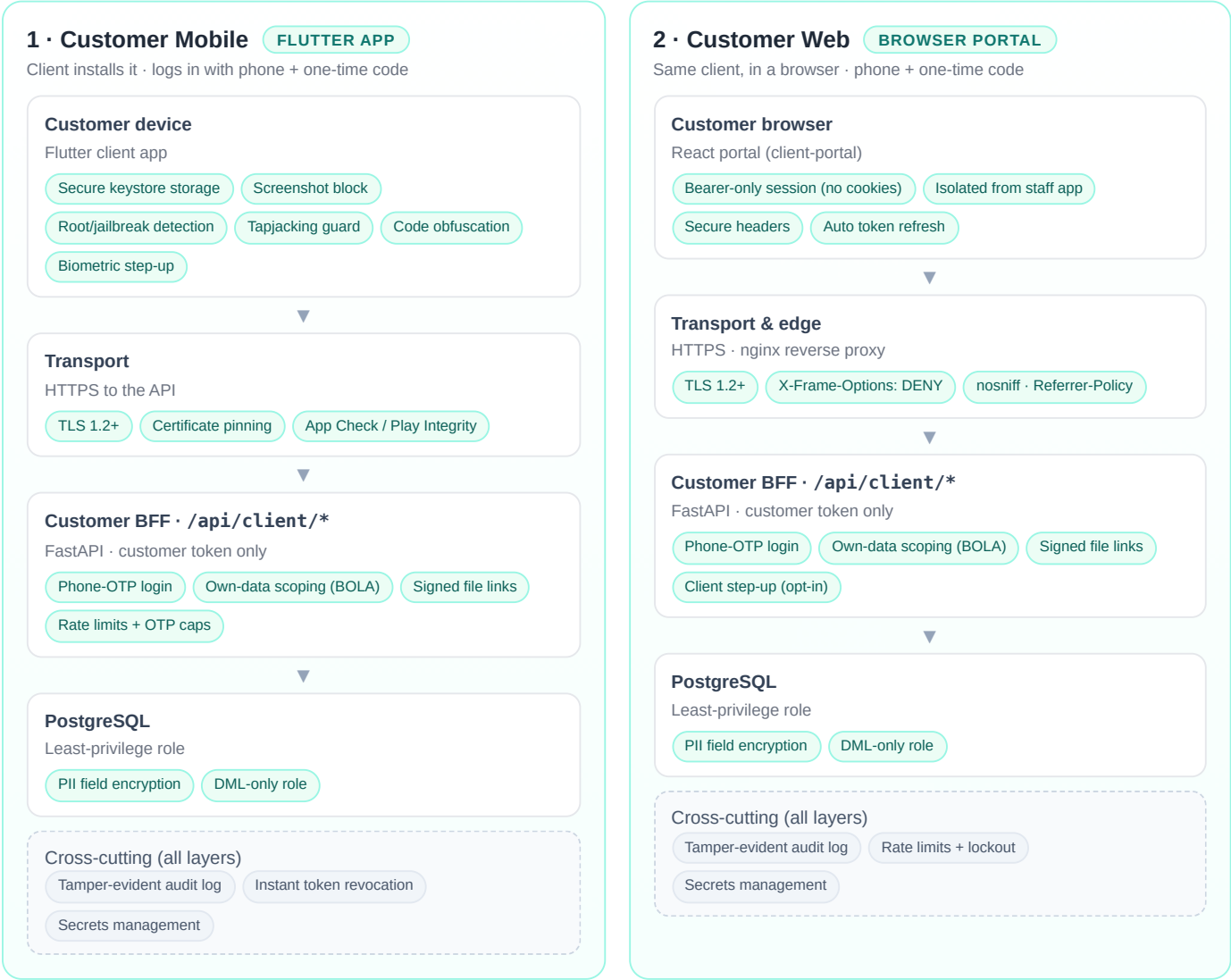
Staff token → only staff routes

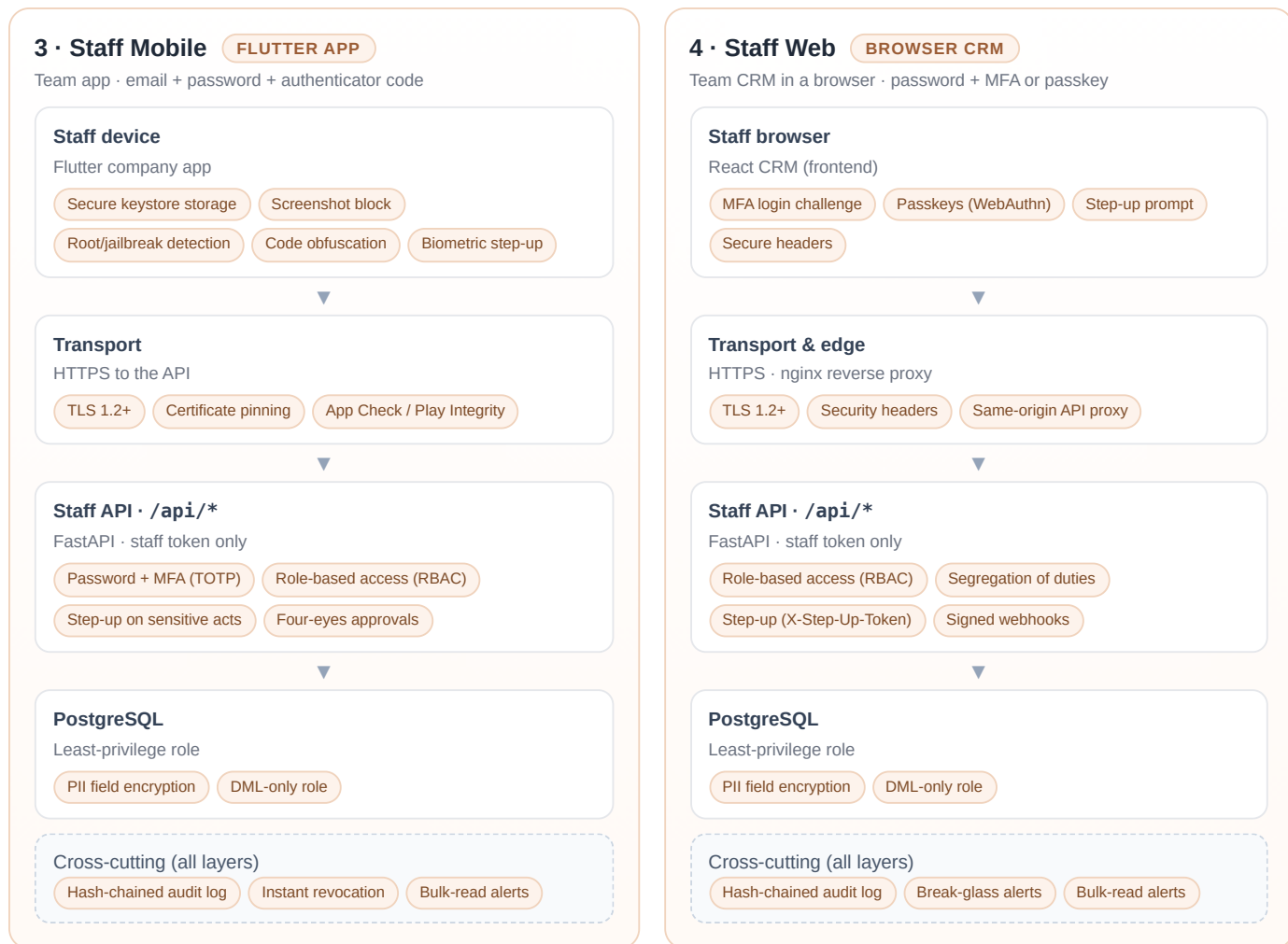
A customer token is **rejected** on staff routes and vice-versa — so one side can never reach the other's data, even if something else goes wrong.

How to read the diagrams: each app is shown as a stack of layers a request passes through — from the user's device/browser at the top, down to the database. Security controls that live at each layer are shown as chips. A dashed band at the bottom lists controls that protect every layer.

Maturity legend (used throughout) ● implemented & active · ● implemented, needs a production setting to switch on (a key, a certificate, HTTPS, or an on/off flag) — not applicable to this app

Most controls ship **off by default** and are turned on during roll-out — deliberately, so nothing is half-configured. This document is honest about what is on versus what still needs a setting.





Tag key: **client** lives in the app on the device/browser · **backend** enforced by the shared API/DB and therefore protects this app too.

1 · Customer Mobile app

- **Phone one-time-code (OTP) login** **backend** — no password to steal; a code is only sent to a number already on file.
- **Certificate pinning & app attestation** **client** — the app only trusts our real server and the backend only accepts our genuine app.
- **On-device hardening** **client** — secure keystore for tokens, screenshot/recording block on sensitive screens, root/jailbreak detection, tapjacking guard, code obfuscation.
- **Biometric step-up** **client** — fingerprint/face confirmation before high-risk actions (accepting an estimate, approving a design).
- **Own-data scoping** **backend** — a customer can only ever see their own projects, files and payments.
- **Signed, expiring file links** · **PII encryption** · **least-privilege DB** · **audit log** **backend** .

2 · Customer Web portal

- **Phone OTP login** **backend** — same passwordless login as the mobile app.
- **Bearer-only session isolation** **client** — the portal keeps its login token in its own browser storage and sends *no cookies*, so it can never share a session with the staff site even on the same server.
- **Browser security headers** **client** — anti-clickjacking (X-Frame-Options: DENY), MIME-sniffing off, referrer minimised.
- **Automatic token refresh** **client** — short-lived access token renews silently; a dead session bounces to login.
- **Own-data scoping** · **signed file links** · **PII encryption** · **rate limits** · **audit log** **backend** .

Not applicable in a browser: certificate pinning, screenshot blocking and app attestation (these are device-app controls). Passkeys are staff-only today.

3 · Staff Mobile app

- **Password + Multi-Factor Authentication (TOTP)** **backend** — a second code from an authenticator app on top of the password.
- **Certificate pinning & app attestation** **client** .
- **On-device hardening** **client** — secure keystore, screenshot block, root detection, obfuscation, biometric step-up.
- **Role-based access (RBAC)** **backend** — each role (sales, designer, accounts, admin, CEO...) sees only what its job needs.
- **Segregation of duties / four-eyes & step-up** **backend** — the person who raises a payment can't also approve it; sensitive actions need a fresh check.
- **Bulk-read alerts** · **hash-chained audit** · **instant revocation** · **PII encryption** **backend** .

4 · Staff Web CRM

- **Password + MFA login challenge & self-service enrolment** **client** + **backend** .
- **Passkeys (WebAuthn / FIDO2)** **client** — phishing-resistant fingerprint/face/PIN sign-in; managed in-app. Shown only over HTTPS.
- **Step-up re-authentication** **client** — a "confirm it's you" prompt appears automatically before privileged actions and replays the request once confirmed.
- **RBAC** · **segregation of duties** · **signed payment webhooks** **backend** .
- **Break-glass & bulk-read alerts** · **hash-chained audit** · **PII encryption** **backend** .

04 What each application has — comparison

● active ● implemented, needs a production setting — not applicable. **backend** controls protect every app through the shared API.

Control	Layer	Cust. Mobile	Cust. Web	Staff Mobile	Staff Web
Identity & login					
Phone one-time-code (OTP) login	backend	●	●	—	—
Password + bcrypt hashing	backend	—	—	●	●
Multi-factor auth (authenticator code)	both	—	—	●	●
Passkeys (WebAuthn / FIDO2)	client	—	—	—	●
Biometric step-up (on device)	client	●	—	●	—
Step-up re-auth on sensitive actions	both	●	●	●	●
Instant token revocation + refresh rotation	backend	●	●	●	●
Access control					
Dual-BFF customer/staff boundary	backend	●	●	●	●
Role-based access control (RBAC)	backend	—	—	●	●
Own-data scoping (anti-BOLA)	backend	●	●	●	●
Segregation of duties / four-eyes	backend	—	—	●	●
Data protection					
TLS / HTTPS in transit	both	●	●	●	●
TLS certificate pinning	client	●	—	●	—
PII field-level encryption (phone/email)	backend	●	●	●	●
Signed, expiring document links	backend	●	●	●	●
Upload content (magic-byte) validation	backend	●	●	●	●
Secure on-device token storage	client	●	—	●	—
App / client hardening					
Bearer-only session isolation (no cookies)	client	●	●	—	—
Browser security headers	client	—	●	—	●
Screenshot / screen-record block	client	●	—	●	—
Root/jailbreak detection + obfuscation	client	●	—	●	—
Tapjacking / overlay protection	client	●	—	●	—
App attestation (App Check / Play Integrity)	both	●	—	●	—
Abuse & fraud prevention					
Login brute-force lockout + OTP caps	backend	●	●	●	●
Signed payment webhooks + idempotency	backend	●	●	●	●
Bulk-read / mass-export alerting	backend	—	—	●	●
Monitoring, governance & build					
Tamper-evident (hash-chained) audit log	backend	●	●	●	●
Break-glass (super-admin) alerting	backend	—	—	●	●
Least-privilege database role	backend	●	●	●	●
Secrets mgmt + fail-fast + secret scanning	backend	●	●	●	●
Privacy rights — DPDP consent/export/erasure	backend	●	●	●	●
CI security gates (SAST, deps, secrets, SBOM)	build	●	●	●	●
Automated authorization tests (BOLA/BFLA)	build	●	●	●	●

Reading it: the customer apps deliberately have fewer login controls (passwordless OTP instead of MFA/passkeys) because customers are low-privilege and see only their own data. Staff apps carry heavier controls (MFA, passkeys, four-eyes, bulk-read alerts) because staff are higher-privilege and touch many customers' data.

05 Why each measure exists — in plain English

One line each: what it is, and the bad thing it prevents.

Measure	What it is	Why we have it (the risk it stops)
Phone OTP login	Log in with a one-time code texted to your phone.	No password exists to be stolen, guessed or reused from another leaked site.
Password + MFA	Password <i>plus</i> a rotating code from an authenticator app.	A stolen or guessed staff password alone is not enough to get in.
Passkeys	Sign in with fingerprint/face; no code to type.	Cannot be phished — there's no code or password to trick an employee into handing over.
Biometric / step-up	Re-confirm identity right before a risky action.	A phone left unlocked, or a hijacked session, still can't approve payments or deletes.
Instant revocation	Deactivating or demoting someone logs them out immediately.	A fired or compromised account loses access now, not whenever their token expires.
Dual-BFF boundary	Customer and staff logins are different, non-swappable keys.	A customer can never reach staff-only data, and vice-versa — even if code has a bug.
RBAC (roles)	Each staff role can do only its own job's tasks.	A designer can't touch finances; a junior can't delete accounts. Limits insider damage.
Own-data scoping	You can only load records that belong to you.	Stops "change the ID in the URL to see someone else's project" (a top web flaw, BOLA).
Four-eyes / segregation	Creator and approver of money actions must differ.	Stops one person quietly paying themselves or a fake vendor.
TLS / HTTPS	All traffic is encrypted end to end.	No one on the Wi-Fi or network can read or tamper with data in transit.
Certificate pinning	The app only trusts our exact server certificate.	Blocks fake "look-alike" servers and man-in-the-middle interception.
PII encryption at rest	Phone/email are encrypted inside the database.	If the database is ever stolen or leaked, personal contact details stay unreadable.
Signed file links	Documents open via short-lived, one-off links.	Files aren't sitting at public URLs anyone could find or share forever.
Upload validation	Checks a file's real type, not its name.	Stops malware disguised as an image or PDF from being uploaded.
Secure device storage	Tokens live in the phone's keychain/keystore.	Another app or a lost phone can't simply read the login token off disk.
Screenshot block	Sensitive screens can't be captured.	Prevents shoulder-surfing screenshots and screen-recording malware leaking data.
Bearer-only isolation	The customer portal sends no cookies.	The customer and staff web apps can share a server yet never share a session.
Security headers	Browser rules against framing/sniffing.	Stops clickjacking (invisible overlays) and content-type tricks in the browser.
Root / attestation	Detect tampered devices; verify the genuine app.	The backend refuses calls from fake, cloned or bot versions of the app.
Brute-force lockout	Too many wrong tries locks the attempt.	Stops automated password/OTP guessing.
Signed webhooks	Payment notifications are cryptographically verified.	A fake "payment received" message can't mark an unpaid order as paid.
Bulk-read alert	Flags a staff account reading many records at once.	Early warning of an insider or hijacked account scraping customer data.
Hash-chained audit	Every action is logged in a tamper-evident chain.	You can trust the record — no one can silently edit or delete history.
Least-privilege DB	The app's DB account can't alter the schema.	A compromised app can't drop tables or wreck the database structure.
Secrets management	Keys live in config, never in code; app checks them.	No passwords/keys leak through the code repository; weak setups won't start.
DPDP privacy rights	Customers can view, export and delete their data.	Meets India's DPDP Act and builds trust; avoids regulatory penalties.

Measure	What it is	Why we have it (the risk it stops)
CI security gates	Every code change is auto-scanned before merge.	Catches vulnerable libraries, leaked secrets and risky code early, automatically.
AuthZ tests	Robots continuously try to access others' data.	Proves, on every change, that access rules still hold — no silent regressions.

The controls are built; most remaining work is **switching them on in production** and **hardening the hosting**. This is the honest "what's left" — presented as a prioritised plan.

Pri.	Gap / item	Why it matters (plain English)	Recommended action
P1	Running on HTTP (no HTTPS/domain yet)	Traffic isn't encrypted end-to-end, and passkeys can't switch on without it.	Point a domain at the server, add a free TLS certificate (Let's Encrypt via the existing proxy), enable HSTS.
P1	MFA offered but not enforced for staff	A stolen staff password alone could still log in until MFA is mandatory.	Require MFA (or a passkey) for all staff; set STEP_UP_ENABLED once enrolled.
P1	Database port is exposed to the internet	The database shouldn't be directly reachable from outside.	Firewall port 5432 to trusted IPs only (or close it); use an SSH tunnel for admin.
P2	PII encryption key held in an env file	The key should be as protected as the data it unlocks.	Move the key to a managed KMS/secret manager; define a rotation policy; run the encryption backfill.
P2	Mobile app attestation & cert-pinning not yet keyed	Until configured, the backend can't fully verify the genuine app / block fake servers.	Configure App Check / Play Integrity keys and certificate pins in the mobile builds.
P2	Shared VPS behind a Traefik panel	The CRM shares a host with other tools; less isolation, port contention on reboot.	Give the CRM its own host/VPC or at least dedicated routing; add a WAF + edge rate-limiting (e.g. Cloudflare).
P2	No independent penetration test yet	Internal testing isn't the same as an outside expert trying to break in.	Commission a VAPT of the 4 apps + API against OWASP ASVS / MASVS; fix findings.
P2	Payment gateway controls pending go-live	Signed webhooks + four-eyes are built but inert until the gateway is wired.	Set the webhook secret and confirm/refund thresholds when the gateway is enabled.
P3	Central monitoring / SIEM & incident response	Alerts exist, but not yet aggregated or rehearsed.	Ship audit/security events to a monitoring tool; write and tabletop an incident-response plan.
P3	Backups & disaster-recovery drill	Backups are only proven when a restore is actually tested.	Automate DB backups off-box; schedule a quarterly restore test; set RTO/RPO targets.

P1 = do first (this month) · **P2** = next 1–2 months · **P3** = as you scale. Use this table to structure the CISO conversation — and to ask which order *they'd* tackle it in.

A focused list to pressure-test the design and find the highest-value next steps. Grouped by theme; the ones marked ★ are the highest-leverage for a small team.

A. Threat model & priorities ★

1. Given we hold customers' contact details, designs and payment status, what would you consider our **top three realistic threats** — and are we spending effort in the right places?
2. For a business our size, what's the **minimum viable security bar** you'd insist on before we onboard real customers at scale?
3. Where are we **over-engineering** (spending effort with low payoff) versus **under-investing**?

B. Identity, authentication & sessions ★

1. Should we **mandate MFA/passkeys for all staff** now, and enforce it (not just offer it)? Any exceptions?
2. Is **phone-OTP** a sufficient customer login, or do you want an option for email OTP / social login / step-up for payments? How should we handle **SIM-swap** risk?
3. Are our **token lifetimes and refresh rotation** sensible? Should we add device binding or re-auth on new devices?
4. How should we handle **account recovery** for staff and customers without opening a social-engineering hole?

C. Data protection & privacy (DPDP) ★

1. Our PII encryption uses a key in our secrets store. Do you require a **managed KMS/HSM**, key rotation policy, and separation of duties for key access?
2. What's your guidance on **data retention & minimisation** — how long should we keep leads, chats and documents, and when must we purge?
3. Are we correctly positioned as **Data Fiduciary** under India's DPDP Act? What consent, breach-notification and grievance processes do we still need?
4. Should customer PII beyond phone/email (addresses, designs) also be encrypted at rest?

D. Infrastructure, hosting & network

1. We're on a single VPS with a reverse proxy (Traefik) shared with other tools. Do you recommend **separating the app onto its own host/VPC**, and putting a WAF / rate-limiter (e.g., Cloudflare) in front?
2. What's your baseline for **network segmentation, firewalling and exposed ports** (our database port is currently reachable)?
3. Requirements for **TLS** (min version, HSTS, certificate management/renewal) once we move to a domain?
4. How should we manage **secrets in production** — move from env files to a secrets manager?

E. Application & API security

1. Would you commission an **independent penetration test** of the four apps + API, and how often? Any preferred scope/standard (OWASP ASVS / MASVS level)?
2. Are our **rate-limiting and anti-automation** controls adequate at the edge, not just per-account?
3. How rigorously should we implement **CSP, CORS and anti-CSRF** for the two web apps?
4. For the mobile apps, what's your bar for **certificate pinning, attestation and anti-tamper** — enough as-is, or hardened further?

F. Monitoring, detection & incident response ★

1. We have a tamper-evident audit log and some alerts (break-glass, bulk-read). What **security events** would you want centralised into a SIEM/alerting tool, and what's a realistic setup for a small team?
2. Do we need a written, tested **incident-response plan** and breach-notification runbook? Would you help us tabletop it?
3. What are your expectations for **backups, disaster recovery and RTO/RPO**, and how often should we test restores?
4. How do we detect and respond to a **compromised staff account or insider threat** quickly?

G. Supply chain & secure development

1. We run SAST, dependency, secret scanning and SBOM in CI. Do you want **gated builds, signed artifacts, and a vulnerability-response SLA**?
2. How should we manage **third-party and AI-assisted code** review and approvals before production?
3. Guidance on **dependency and base-image patching cadence**?

H. Payments & fraud

1. As we enable the payment gateway, what **controls beyond signed webhooks and four-eyes** do you expect (limits, anomaly detection, reconciliation)?
2. Any **PCI-DSS scope** we must be mindful of even when using a hosted gateway?

I. Governance, people & compliance

1. What lightweight **security policies** (access, onboarding/offboarding, acceptable use, vendor risk) should a company our size have first?
2. What **security-awareness training** do you recommend for our staff (phishing especially)?

3. Are there **certifications or attestations** (ISO 27001, SOC 2, VAPT report) our future customers/partners will ask for — and when should we start?

One closing question worth asking: "If you had a limited budget to spend on *one* improvement to these four applications in the next 90 days, what would it be?" — it turns the conversation into a concrete, prioritised action.